Vector Databases: Tuning, Scaling and Costs

1. Introduction

Every RAG question runs a search in the vector database. As data and users grow, three questions come up: how to tune the index, when to scale, and what it all costs.

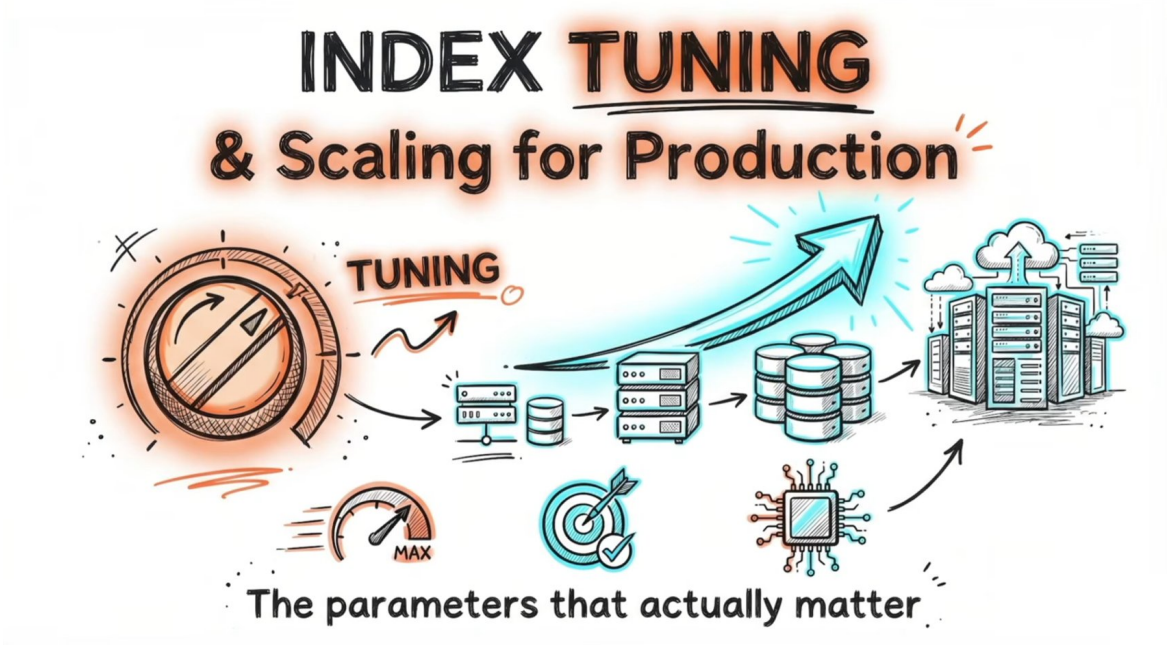


Figure 1: Index tuning and scaling for production.

2. Index Tuning (HNSW)

Comparing the question with every vector, one by one, is exact but slow. Most vector databases use an approximate index called **HNSW** (Hierarchical Navigable Small World). It is a graph where each vector links to its closest neighbours, and a search jumps through the graph towards the best matches. It is very fast and almost always finds the right results.

Three parameters control it:

- **M (max connections):** how many links each vector keeps. More links give better accuracy, but a bigger index and more RAM.
- **ef_construction:** how much effort goes into building the graph. Higher gives a better graph, but a slower build.
- **ef_search:** how many candidates each search checks. Higher gives better accuracy, but slower search.

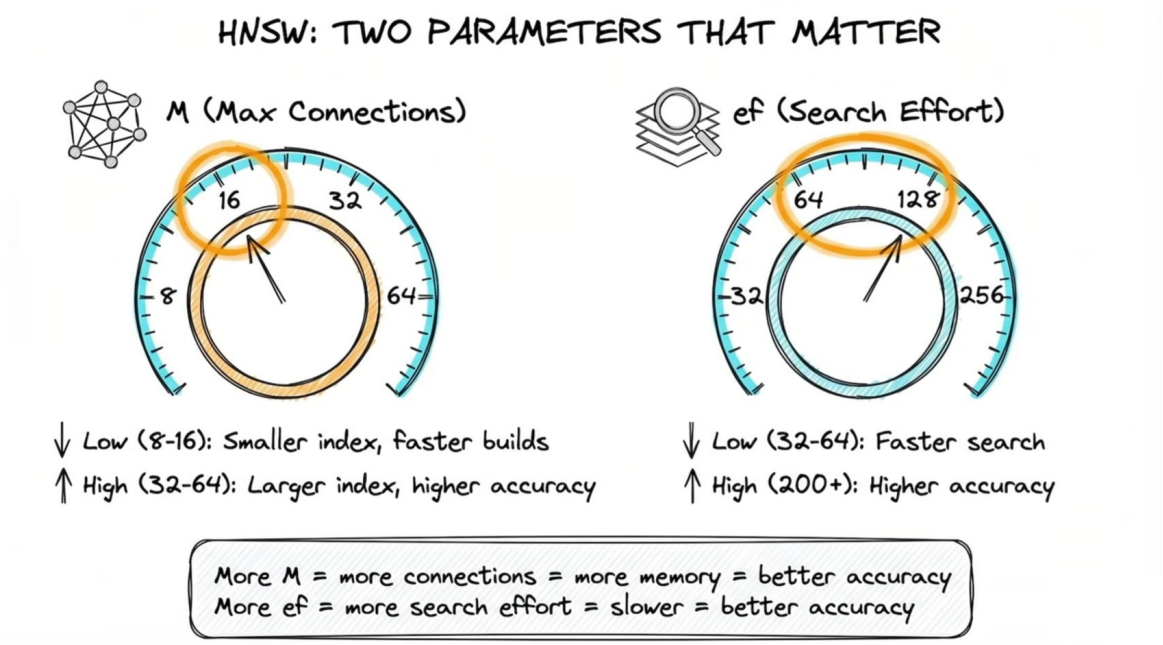


Figure 2: M and ef: the two dials of HNSW. Here "ef" is the search effort, ef_search.

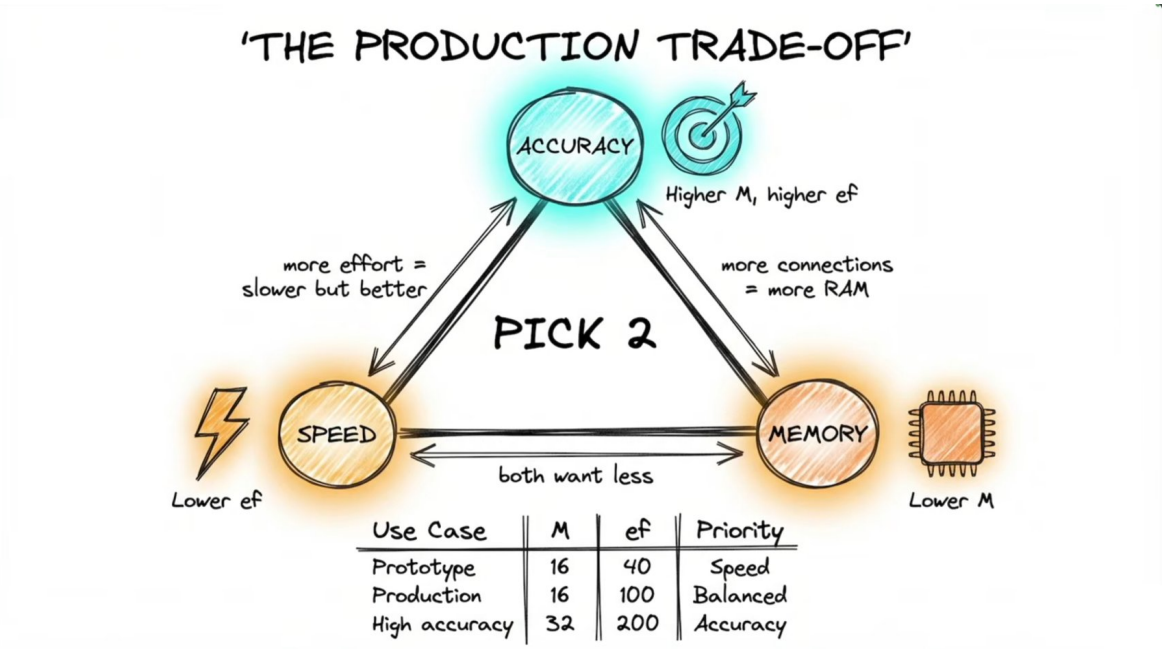


Figure 3: We trade accuracy against speed and memory.

> **Tip:** pgvector starts with m=16, ef_construction=64 and ef_search=40, a good default. Tune ef_search first: it needs no rebuild. To choose values, compare the results with an exact search on test questions (recall@k).

3. Scaling

First, a quick memory estimate: **vectors x dimensions x 4 bytes**. One million vectors of 1536 dimensions take about 6 GB, plus the index on top.

- **Vertical (scale up):** a bigger machine with more RAM and CPU. Simple and no code changes, so try it first.
- **Horizontal (sharding):** split the data across machines. Each query goes to every shard and the results are merged. It scales further, but it is complex.

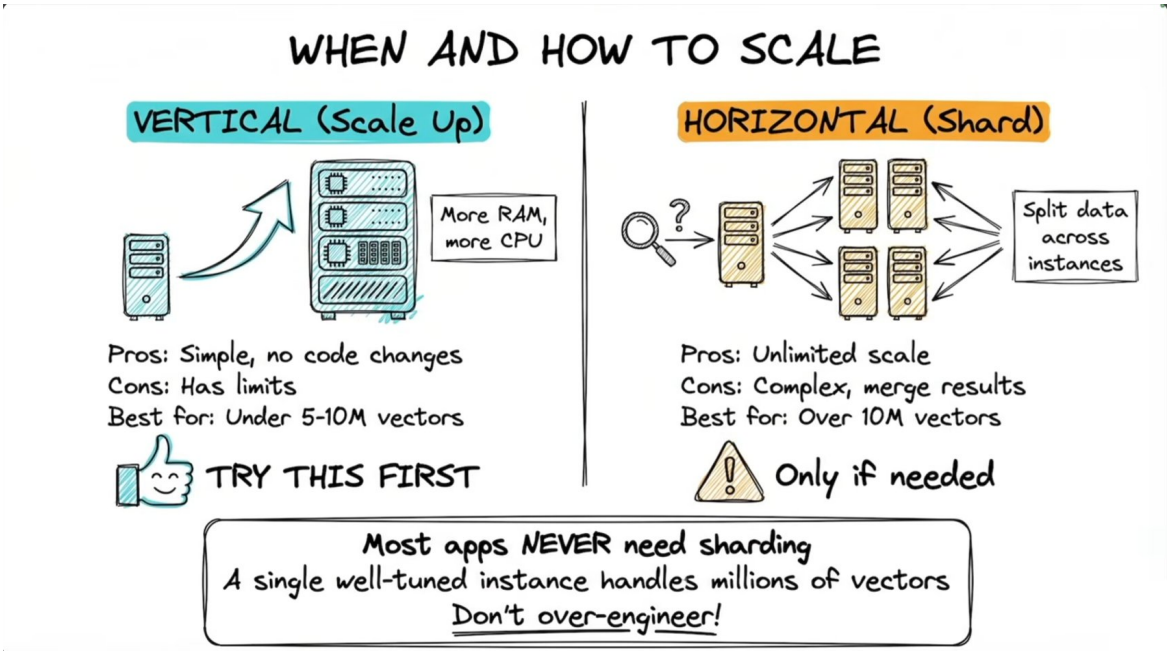


Figure 4: Scale up first; shard only if needed.

Most apps never need sharding. A single, well-tuned instance handles millions of vectors.

4. Managed vs Self-Hosted

- **Managed (e.g. Pinecone):** the provider runs the database. Scaling is automatic and there is no ops work, but it costs more at scale and gives less control.
- **Self-hosted (e.g. pgvector):** we run it inside our own PostgreSQL. It is cheaper at scale and gives full control, but backups, updates and monitoring are our job.
- **In between:** a managed PostgreSQL (e.g. AWS RDS) with pgvector. The cloud runs the server, we keep PostgreSQL.

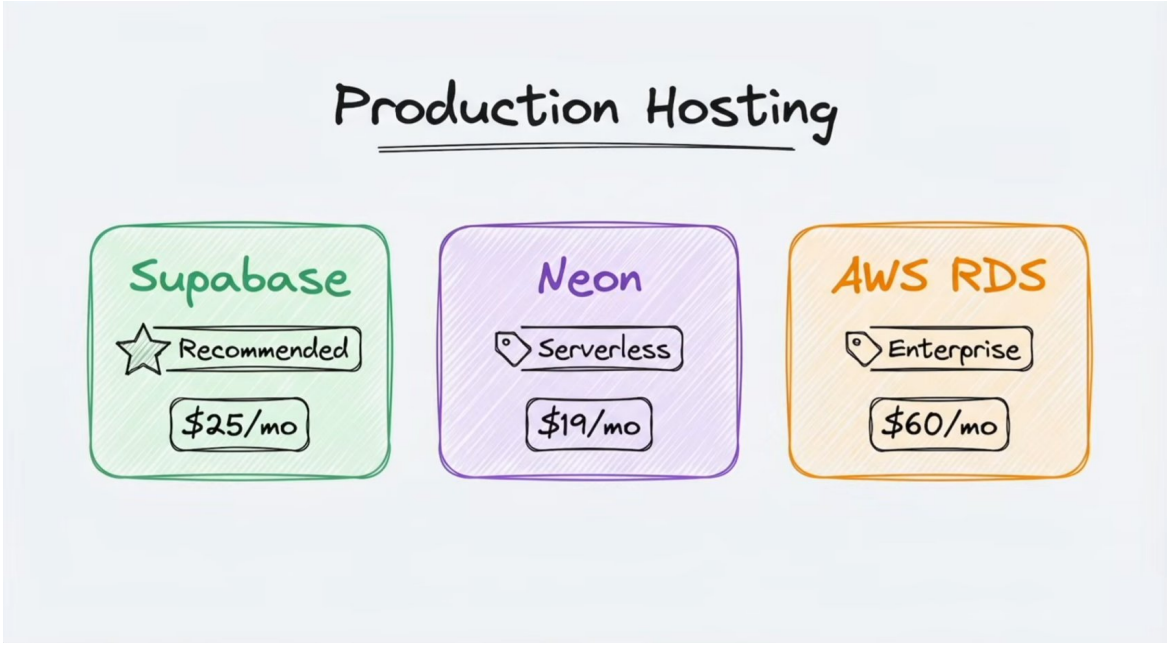


Figure 5: Managed PostgreSQL services that support pgvector. Prices are examples and change often.

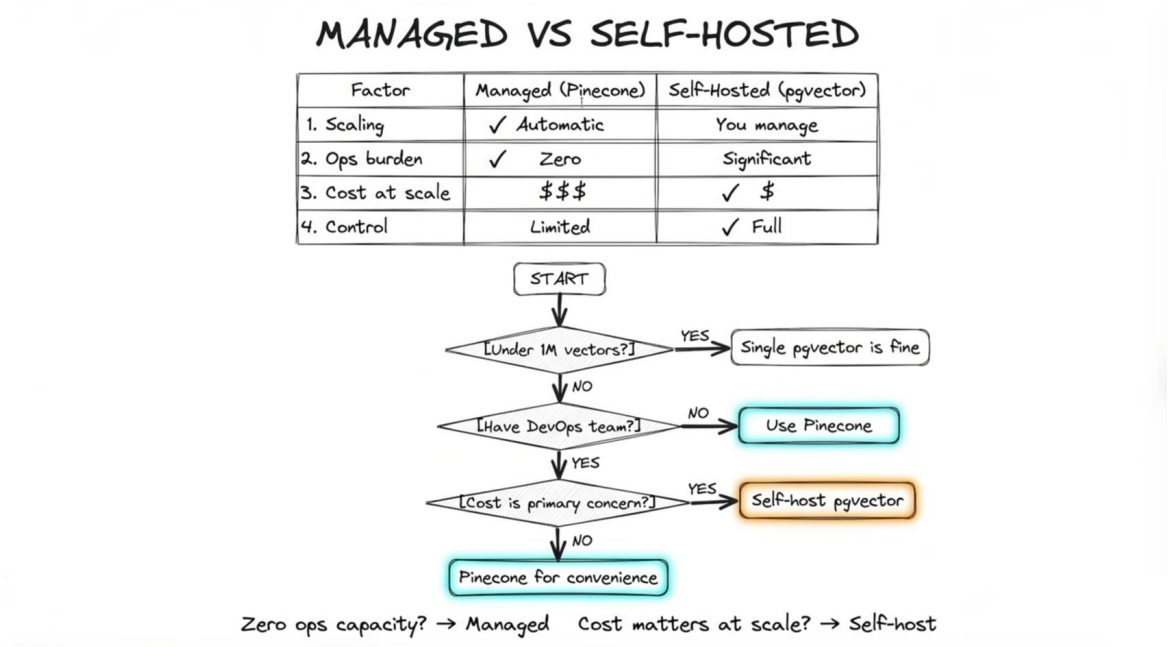


Figure 6: Managed vs self-hosted, and a simple way to decide.

5. Costs

At small scale the options cost about the same, so convenience wins. As data grows, the prices move apart and self-hosting starts to pay off.

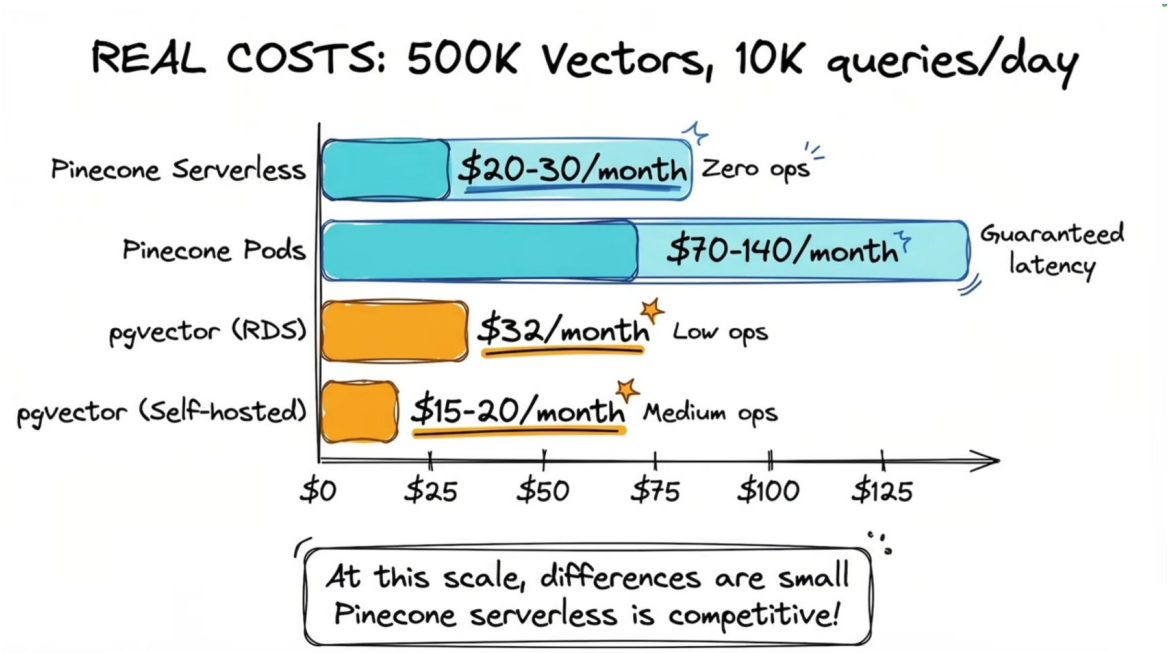


Figure 7: At 500K vectors the differences are small.

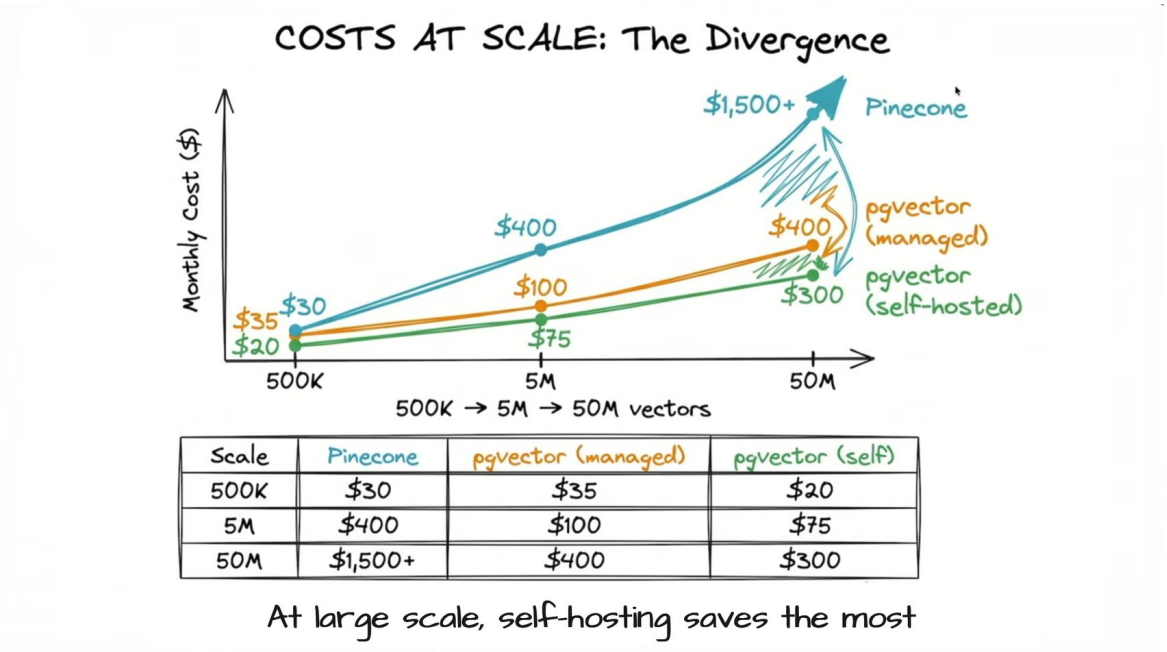


Figure 8: At millions of vectors, the costs move apart.

> **Careful:** these prices are examples and change often; check each provider's current pricing. Also count our own time: self-hosting is cheaper only when the monthly savings are bigger than the time we spend running it.

6. Cost Optimization

- **Reduce dimensions:** some models (e.g. OpenAI text-embedding-3) can return shorter vectors. 1536 to 512 dimensions is 3 times less storage, with a small accuracy loss. Test it.
- **Quantization:** store each number in fewer bits. float32 to int8 cuts vector memory by 75% (4 bytes to 1), with little accuracy loss.
- **Batch requests:** send many texts in one call instead of one call per text.
- **Caching:** keep the results of frequent questions, so repeated questions cost almost nothing.
- **Right-size:** pay for the machine we need now, not the one we might need one day.

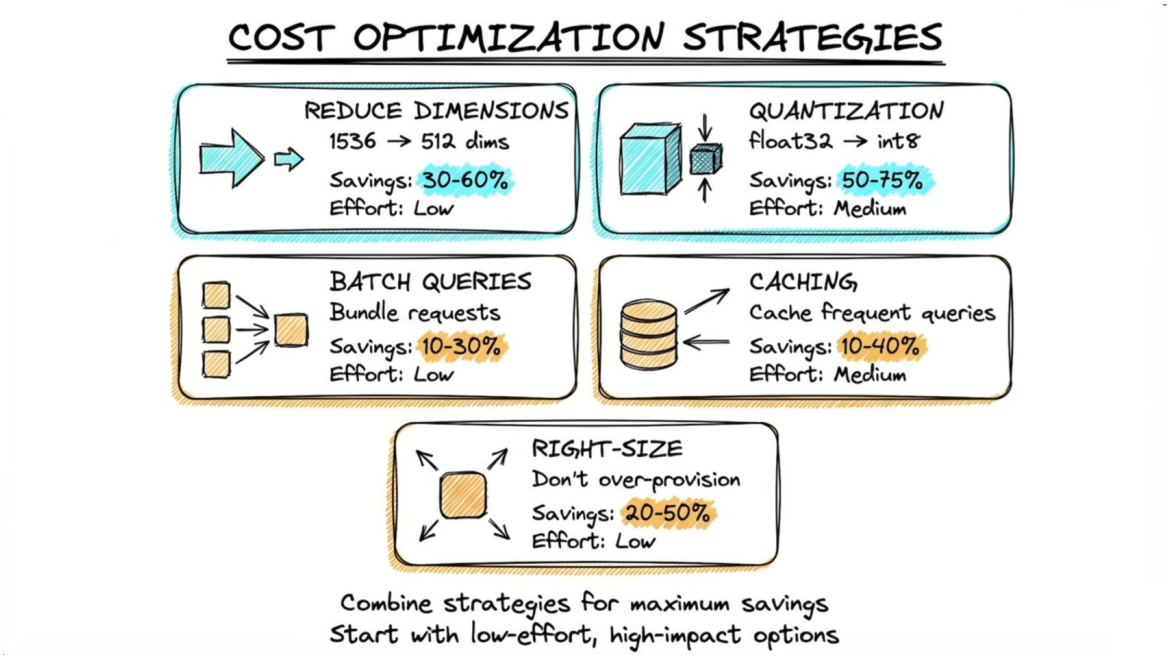


Figure 9: Ways to cut costs. The savings are rough; measure on your own data.

7. Conclusion

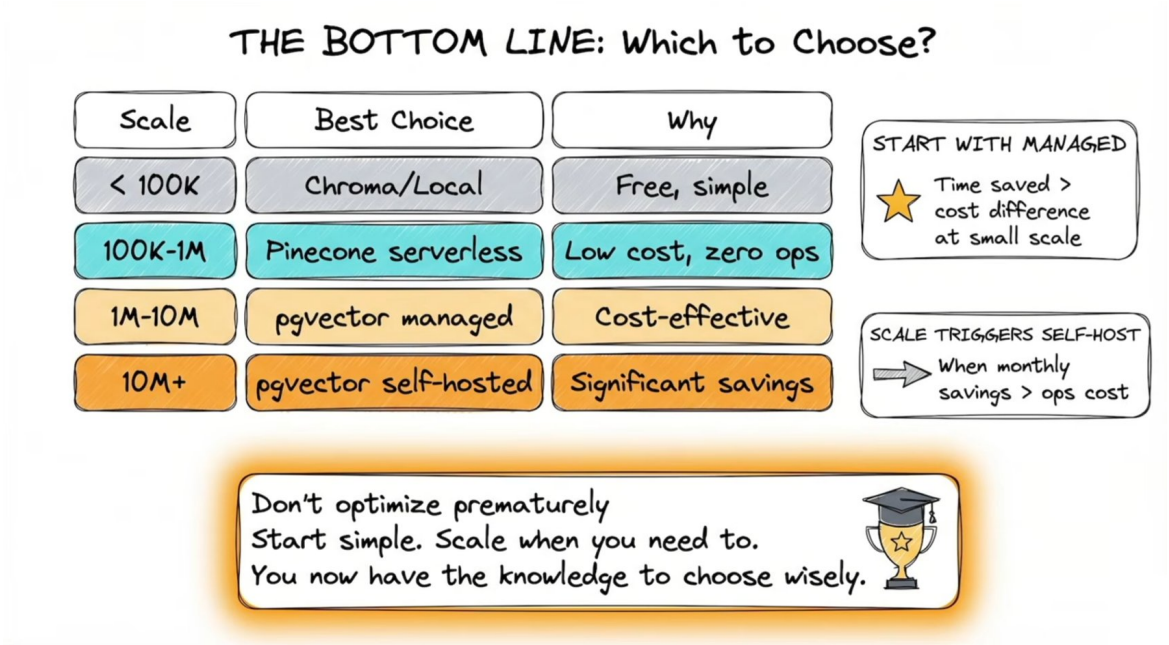


Figure 10: Which option fits which scale.

Example: two real cases

- **Local accounting office:** about 5,000 documents (tax circulars, laws, procedures, client FAQs), around 25,000 pages. In chunks of about 500 tokens this gives **about 30,000 vectors**, only 0.2 GB of memory. Embedding everything once with text-embedding-3-small ($0.02 per 1M tokens) costs well under $1, even for Greek text. Chroma on the office server, or pgvector in an existing PostgreSQL, is enough for years. Nothing to tune or scale.
- **Medium online store:** 20,000 products and 300,000 customer reviews give **about 320,000 vectors**, around 2 GB at 1536 dimensions. Start with pgvector in the PostgreSQL the shop already uses, or a managed service for zero ops (tens of dollars per month). If the catalog grows to millions, or search gets slow, we go step by step: raise ef_search, then

reduce dimensions or quantize (2 GB becomes about 0.5 GB), then a bigger machine. Shard only after all of these.

> **Note:** in both cases the embeddings cost almost nothing and the database is cheap. The main running cost is the LLM that writes the answers.

Start simple: Chroma or a single pgvector for small data, a managed service when we want zero ops, and self-hosting when the savings are bigger than the ops work. Tune before we scale, and scale up before we shard.